

Use SSH keys to communicate with GitLab (FREE)

Git is a distributed version control system, which means you can work locally, then share or *push* your changes to a server. In this case, the server you push to is GitLab.

GitLab uses the SSH protocol to securely communicate with Git. When you use SSH keys to authenticate to the GitLab remote server, you don't need to supply your username and password each time.

Prerequisites

To use SSH to communicate with GitLab, you need:

- The OpenSSH client, which comes pre-installed on GNU/Linux, macOS, and Windows 10.
- SSH version 6.5 or later. Earlier versions used an MD5 signature, which is not secure.

To view the version of SSH installed on your system, run `ssh -V`.

Supported SSH key types

To communicate with GitLab, you can use the following SSH key types:

- [ED25519](#)
- [ED25519_SK](#) (Available in GitLab 14.8 and later.)
- [ECDSA_SK](#) (Available in GitLab 14.8 and later.)
- [RSA](#)
- DSA ([Deprecated](#) in GitLab 11.0.)
- ECDSA (As noted in [Practical Cryptography With Go](#), the security issues related to DSA also apply to ECDSA.)

Administrators can [restrict which keys are permitted and their minimum lengths](#).

ED25519 SSH keys

The book [Practical Cryptography With Go](#) suggests that [ED25519](#) keys are more secure and performant than RSA keys.

OpenSSH 6.5 introduced ED25519 SSH keys in 2014, and they should be available on most operating systems.

ED25519_SK SSH keys

[Introduced](#) in GitLab 14.8.

To use ED25519_SK SSH keys on GitLab, your local client and GitLab server must have [OpenSSH 8.2](#) or later installed.

ECDSA_SK SSH keys

[Introduced](#) in GitLab 14.8.

To use ECDSA_SK SSH keys on GitLab, your local client and GitLab server must have [OpenSSH 8.2](#) or later installed.

RSA SSH keys

Available documentation suggests ED25519 is more secure than RSA.

If you use an RSA key, the US National Institute of Science and Technology in [Publication 800-57 Part 3 \(PDF\)](#) recommends a key size of at least 2048 bits. The default key size depends on your version of `ssh-keygen`. Review the `man` page for your installed `ssh-keygen` command for details.

See if you have an existing SSH key pair

Before you create a key pair, see if a key pair already exists.

1. Go to your home directory.
2. Go to the `.ssh/` subdirectory. If the `.ssh/` subdirectory doesn't exist, you are either not in the home directory, or you haven't used `ssh` before. In the latter case, you need to [generate an SSH key pair](#).
3. See if a file with one of the following formats exists:

Algorithm	Public key	Private key
ED25519 (preferred)	id_ed25519.pub	id_ed25519
ED25519_SK	id_ed25519_sk.pub	id_ed25519_sk
ECDSA_SK	id_ecdsa_sk.pub	id_ecdsa_sk
RSA (at least 2048-bit key size)	id_rsa.pub	id_rsa
DSA (deprecated)	id_dsa.pub	id_dsa
ECDSA	id_ecdsa.pub	id_ecdsa

Generate an SSH key pair

If you do not have an existing SSH key pair, generate a new one:

1. Open a terminal.
2. Run `ssh-keygen -t` followed by the key type and an optional comment. This comment is included in the `.pub` file that's created. You may want to use an email address for the comment.

For example, for ED25519:

```
ssh-keygen -t ed25519 -C "<comment>"
```

For 2048-bit RSA:

```
ssh-keygen -t rsa -b 2048 -C "<comment>"
```

3. Press `Enter`. Output similar to the following is displayed:

```
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/user/.ssh/id_ed25519):
```

4. Accept the suggested filename and directory, unless you are generating a [deploy key](#) or want to save in a specific directory where you store other keys.

You can also dedicate the SSH key pair to a [specific host](#).

5. Specify a [passphrase](#):

```
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:
```

A confirmation is displayed, including information about where your files are stored.

A public and private key are generated. [Add the public SSH key to your GitLab account](#) and keep the private key secure.

Configure SSH to point to a different directory

If you did not save your SSH key pair in the default directory, configure your SSH client to point to the directory where the private key is stored.

1. Open a terminal and run this command:

```
eval $(ssh-agent -s)  
ssh-add <directory to private SSH key>
```

2. Save these settings in the `~/.ssh/config` file. For example:

```
# GitLab.com  
Host gitlab.com
```

```
PreferredAuthentications publickey
IdentityFile ~/.ssh/gitlab_com_rsa

# Private GitLab instance
Host gitlab.company.com
PreferredAuthentications publickey
IdentityFile ~/.ssh/example_com_rsa
```

For more information on these settings, see the [man ssh_config](#) page in the SSH configuration manual.

Public SSH keys must be unique to GitLab because they bind to your account. Your SSH key is the only identifier you have when you push code with SSH. It must uniquely map to a single user.

Update your SSH key passphrase

You can update the passphrase for your SSH key:

1. Open a terminal and run this command:

```
ssh-keygen -p -f /path/to/ssh_key
```

2. At the prompts, enter the passphrase and then press .

Upgrade your RSA key pair to a more secure format

If your version of OpenSSH is between 6.5 and 7.8, you can save your private RSA SSH keys in a more secure OpenSSH format by opening a terminal and running this command:

```
ssh-keygen -o -f ~/.ssh/id_rsa
```

Alternatively, you can generate a new RSA key with the more secure encryption format with the following command:

```
ssh-keygen -o -t rsa -b 4096 -C "<comment>"
```

Generate an SSH key pair for a FIDO/U2F hardware security key

To generate ED25519_SK or ECDSA_SK SSH keys, you must use OpenSSH 8.2 or later:

1. Insert a hardware security key into your computer.
2. Open a terminal.
3. Run `ssh-keygen -t` followed by the key type and an optional comment. This comment is included in the `.pub` file that's created. You may want to use an email address for the comment.

For example, for ED25519_SK:

```
ssh-keygen -t ed25519-sk -C "<comment>"
```

For ECDSA_SK:

```
ssh-keygen -t ecdsa-sk -C "<comment>"
```

If your security key supports FIDO2 resident keys, you can enable this when creating your SSH key:

```
ssh-keygen -t ed25519-sk -O resident -C "<comment>"
```

`-O resident` indicates that the key should be stored on the FIDO authenticator itself. Resident key is easier to import to a new computer because it can be loaded directly from the security key by [ssh-add -K](#) or [ssh-keygen -K](#).

4. Press . Output similar to the following is displayed:

```
Generating public/private ed25519-sk key pair.
```

You may need to touch your authenticator to authorize key generation.

- 5. Touch the button on the hardware security key.
- 6. Accept the suggested filename and directory:

Enter file in which to save the key (/home/user/.ssh/id_ed25519_sk):

- 7. Specify a [passphrase](#):

Enter passphrase (empty for no passphrase):
Enter same passphrase again:

A confirmation is displayed, including information about where your files are stored.

A public and private key are generated. [Add the public SSH key to your GitLab account](#).

Add an SSH key to your GitLab account

[Introduced](#) in GitLab 15.4, default expiration date suggested in UI.

To use SSH with GitLab, copy your public key to your GitLab account:

- 1. Copy the contents of your public key file. You can do this manually or use a script. For example, to copy an ED25519 key to the clipboard:

macOS

```
tr -d '\n' < ~/.ssh/id_ed25519.pub | pbcopy
```

Linux (requires the xclip package)

```
xclip -sel clip < ~/.ssh/id_ed25519.pub
```

Git Bash on Windows

```
cat ~/.ssh/id_ed25519.pub | clip
```

Replace `id_ed25519.pub` with your filename. For example, use `id_rsa.pub` for RSA.

- 2. Sign in to GitLab.
- 3. On the top bar, in the top right corner, select your avatar.
- 4. Select **Preferences**.
- 5. On the left sidebar, select **SSH Keys**.
- 6. In the **Key** box, paste the contents of your public key. If you manually copied the key, make sure you copy the entire key, which starts with `ssh-rsa`, `ssh-dss`, `ecdsa-sha2-nistp256`, `ecdsa-sha2-nistp384`, `ecdsa-sha2-nistp521`, `ssh-ed25519`, `sk-ecdsa-sha2-nistp256@openssh.com`, or `sk-ssh-ed25519@openssh.com`, and may end with a comment.
- 7. In the **Title** box, type a description, like `Work Laptop` or `Home Workstation`.
- 8. Optional. Update **Expiration date** to modify the default expiration date. In:
 - GitLab 13.12 and earlier, the expiration date is informational only. It doesn't prevent you from using the key. Administrators can view expiration dates and use them for guidance when [deleting keys](#).
 - GitLab checks all SSH keys at 02:00 AM UTC every day. It emails an expiration notice for all SSH keys that expire on the current date. ([Introduced](#) in GitLab 13.11.)
 - GitLab checks all SSH keys at 01:00 AM UTC every day. It emails an expiration notice for all SSH keys that are scheduled to expire seven days from now. ([Introduced](#) in GitLab 13.11.)

9. Select **Add key**.

Verify that you can connect

Verify that your SSH key was added correctly.

The following commands use the example hostname `gitlab.example.com`. Replace this example hostname with your GitLab instance's hostname, for example, `git@gitlab.com`.

1. To ensure you're connecting to the correct server, check the server's SSH host keys fingerprint. For:
 - GitLab.com, see the [SSH host keys fingerprints](#) documentation.
 - GitLab.com or another GitLab instance, see `gitlab.example.com/help/instance_configuration#ssh-host-keys-fingerprints` where `gitlab.example.com` is `gitlab.com` (for GitLab.com) or the address of the GitLab instance.
2. Open a terminal and run this command, replacing `gitlab.example.com` with your GitLab instance URL:

```
ssh -T git@gitlab.example.com
```

3. If this is the first time you connect, you should verify the authenticity of the GitLab host. If you see a message like:

```
The authenticity of host 'gitlab.example.com (35.231.145.151)' can't be established.  
ECDSA key fingerprint is SHA256:HbW3g8zUjNSksFbqTiUWPWg2Bq1x8xdGUrliXFzSnUw.  
Are you sure you want to continue connecting (yes/no)? yes  
Warning: Permanently added 'gitlab.example.com' (ECDSA) to the list of known hosts.
```

Type `yes` and press `Enter`.

4. Run the `ssh -T git@gitlab.example.com` command again. You should receive a *Welcome to GitLab, @username !* message.

If the welcome message doesn't appear, you can troubleshoot by running `ssh` in verbose mode:

```
ssh -Tvvv git@gitlab.example.com
```

Use different keys for different repositories

You can use a different key for each repository.

Open a terminal and run this command:

```
git config core.sshCommand "ssh -o IdentitiesOnly=yes -i ~/.ssh/private-key-filename-for-this-repository -F /dev/null"
```

This command does not use the SSH Agent and requires Git 2.10 or later. For more information on `ssh` command options, see the `man` pages for both `ssh` and `ssh_config`.

Use different accounts on a single GitLab instance

You can use multiple accounts to connect to a single instance of GitLab. You can do this by using the command in the [previous topic](#). However, even if you set `IdentitiesOnly` to `yes`, you cannot sign in if an `IdentityFile` exists outside of a `Host` block.

Instead, you can assign aliases to hosts in the `~/.ssh/config` file.

- For the `Host`, use an alias like `user_1.gitlab.com` and `user_2.gitlab.com`. Advanced configurations are more difficult to maintain, and these strings are easier to understand when you use tools like `git remote`.
- For the `IdentityFile`, use the path the private key.

```
# User1 Account Identity  
Host <user_1.gitlab.com>  
  Hostname gitlab.com  
  PreferredAuthentications publickey  
  IdentityFile ~/.ssh/<example_ssh_key1>  
  
# User2 Account Identity  
Host <user_2.gitlab.com>
```

```
Hostname gitlab.com
PreferredAuthentications publickey
IdentityFile ~/.ssh/<example_ssh_key2>
```

Now, to clone a repository for `user_1`, use `user_1.gitlab.com` in the `git clone` command:

```
git clone git@<user_1.gitlab.com>:gitlab-org/gitlab.git
```

To update a previously-cloned repository that is aliased as `origin` :

```
git remote set-url origin git@<user_1.gitlab.com>:gitlab-org/gitlab.git
```

NOTE: Private and public keys contain sensitive data. Ensure the permissions on the files make them readable to you but not accessible to others.

Configure two-factor authentication (2FA)

You can set up two-factor authentication (2FA) for [Git over SSH](#). We recommend using [ED25519 SK](#) or [ECDSA SK](#) SSH keys.

Use EGit on Eclipse

If you are using [EGit](#), you can [add your SSH key to Eclipse](#).

Use SSH on Microsoft Windows

If you're running Windows 10, you can either use the [Windows Subsystem for Linux \(WSL\)](#), with [WSL 2](#) which has both `git` and `ssh` preinstalled, or install [Git for Windows](#) to use SSH through PowerShell.

The SSH key generated in WSL is not directly available for Git for Windows, and vice versa, as both have a different home directory:

- WSL: `/home/<user>`
- Git for Windows: `C:\Users\<user>`

You can either copy over the `.ssh/` directory to use the same key, or generate a key in each environment.

If you're running Windows 11 and using [OpenSSH for Windows](#), ensure the `HOME` environment variable is set correctly. Otherwise, your private SSH key might not be found.

Alternative tools include:

- [Cygwin](#)
- [PuttyGen](#)

Overriding SSH settings on the GitLab server

GitLab integrates with the system-installed SSH daemon and designates a user (typically named `git`) through which all access requests are handled. Users who connect to the GitLab server over SSH are identified by their SSH key instead of their username.

SSH *client* operations performed on the GitLab server are executed as this user. You can modify this SSH configuration. For example, you can specify a private SSH key for this user to use for authentication requests. However, this practice is **not supported** and is strongly discouraged as it presents significant security risks.

GitLab checks for this condition, and directs you to this section if your server is configured this way. For example:

```
$ gitlab-rake gitlab:check

Git user has default SSH configuration? ... no
Try fixing it:
mkdir ~/gitlab-check-backup-1504540051
sudo mv /var/lib/git/.ssh/id_rsa ~/gitlab-check-backup-1504540051
sudo mv /var/lib/git/.ssh/id_rsa.pub ~/gitlab-check-backup-1504540051
For more information see:
doc/user/ssh.md#overriding-ssh-settings-on-the-gitlab-server
Please fix the error above and rerun the checks.
```

Remove the custom configuration as soon as you can. These customizations are **explicitly not supported** and may stop working at any time.

Troubleshooting

Password prompt with `git clone`

When you run `git clone`, you may be prompted for a password, like `git@gitlab.example.com's password: .` This indicates that something is wrong with your SSH setup.

- Ensure that you generated your SSH key pair correctly and added the public SSH key to your GitLab profile.
- Try to manually register your private SSH key by using `ssh-agent`.
- Try to debug the connection by running `ssh -Tv git@example.com`. Replace `example.com` with your GitLab URL.
- Ensure you followed all the instructions in [Use SSH on Microsoft Windows](#).

Could not resolve hostname error

You may receive the following error when [verifying that you can connect](#):

```
ssh: Could not resolve hostname gitlab.example.com: nodename nor servname provided, or not known
```

If you receive this error, restart your terminal and try the command again.

Key enrollment failed: invalid format error

You may receive the following error when [generating an SSH key pair for a FIDO/U2F hardware security key](#):

```
Key enrollment failed: invalid format
```

You can troubleshoot this by trying the following:

- Run the `ssh-keygen` command using `sudo`.
- Verify your IDO/U2F hardware security key supports the key type provided.
- Verify the version of OpenSSH is 8.2 or greater by running `ssh -v`.